

Artificial Intelligence

Prof. Chang-Hee Han

7

Working with Keras: A deep dive (2)

7.0 This chapter covers

▣ Today lecture covers

- Using built-in Keras training and evaluation loops
- Using Keras callbacks to customize training
- Using TensorBoard to monitor training and evaluation metrics
- Writing training and evaluation loops from scratch

7.3 Using built-in training and evaluation loops

▣ Using built-in training and evaluation loops

- The idea of gradually exposing complexity—from very simple to very flexible, step by step across the whole workflow—also applies to model training.
- Keras provides many different workflows for training models.
- You can easily call the fit() method with data, or write your own advanced training algorithm from scratch.

7.3 Using built-in training and evaluation loops

The standard workflow

- You are already familiar with the compile(), fit(), evaluate(), predict() workflow.

Listing 7.17 The standard workflow: `compile()`, `fit()`, `evaluate()`, `predict()`

```
from tensorflow.keras.datasets import mnist

def get_mnist_model():
    inputs = keras.Input(shape=(28 * 28,))
    features = layers.Dense(512, activation="relu")(inputs)
    features = layers.Dropout(0.5)(features)
    outputs = layers.Dense(10, activation="softmax")(features)
    model = keras.Model(inputs, outputs)
    return model

(images, labels), (test_images, test_labels) = mnist.load_data()
images = images.reshape((60000, 28 * 28)).astype("float32") / 255
test_images = test_images.reshape((10000, 28 * 28)).astype("float32") / 255
train_images, val_images = images[10000:], images[:10000]
train_labels, val_labels = labels[10000:], labels[:10000]

model = get_mnist_model()
model.compile(optimizer="rmsprop",
              loss="sparse_categorical_crossentropy",
              metrics=["accuracy"])
model.fit(train_images, train_labels,
          epochs=3,
          validation_data=(val_images, val_labels))
test_metrics = model.evaluate(test_images, test_labels)
predictions = model.predict(test_images)
```

Create a model (we factor this into a separate function so as to reuse it later).

Load your data, reserving some for validation.

Compile the model by specifying its optimizer, the loss function to minimize, and the metrics to monitor.

Use fit() to train the model, optionally providing validation data to monitor performance on unseen data.

Use evaluate() to compute the loss and metrics on new data.

Use predict() to compute classification probabilities on new data.

7.3 Using built-in training and evaluation loops

▣ Using built-in training and evaluation loops

- There are a couple of ways you can customize this simple workflow:
  - ➢ Provide your own custom metrics.
  - ➢ Pass callbacks to the fit() method to schedule actions to be taken at specific points during training.
- Let’s take a look at these.

7.3 Using built-in training and evaluation loops

▣ 7.3.1 Writing your own metrics

- A metric is the key to measuring a model's performance.
- It is especially important to measure the gap between training and test performance.
- Common metrics for classification and regression are already included in the keras.metrics module.
- In most cases, you can use these built-in metrics.
- For uncommon tasks, you should be able to create your own custom metric.
- A Keras metric is a class that inherits from keras.metrics.Metric.
- Like layers, metrics store internal state in TensorFlow variables.
- Unlike layers, these variables are not updated through backpropagation.
- You must define the state update logic directly in the update_state() method.

7.3 Using built-in training and evaluation loops

▣ 7.3.1 Writing your own metrics

- For example, here's a simple custom metric that measures the root mean squared error (RMSE).

Listing 7.18 Implementing a custom metric by subclassing the `Metric` class

```
import tensorflow as tf

class RootMeanSquaredError(keras.metrics.Metric):

    def __init__(self, name="rmse", **kwargs):
        super().__init__(name=name, **kwargs)
        self.mse_sum = self.add_weight(name="mse_sum", initializer="zeros")
        self.total_samples = self.add_weight(
            name="total_samples", initializer="zeros", dtype="int32")

    def update_state(self, y_true, y_pred, sample_weight=None):
        y_true = tf.one_hot(y_true, depth=tf.shape(y_pred)[1])
        mse = tf.reduce_sum(tf.square(y_true - y_pred))
        self.mse_sum.assign_add(mse)
        num_samples = tf.shape(y_pred)[0]
        self.total_samples.assign_add(num_samples)
```

Subclass the Metric class.

Define the state variables in the constructor. Like for layers, you have access to the add_weight() method.

To match our MNIST model, we expect categorical predictions and integer labels.

Implement the state update logic in update_state(). The y_true argument is the targets (or labels) for one batch, while y_pred represents the corresponding predictions from the model. You can ignore the sample_weight argument—we won't use it here.

7.3 Using built-in training and evaluation loops

▣ 7.3.1 Writing your own metrics

- You use the result() method to return the current value of the metric:

```
def result(self):
    return tf.sqrt(self.mse_sum / tf.cast(self.total_samples, tf.float32))
```

- Meanwhile, you also need to expose a way to reset the metric state without having to reinstantiate it—this enables the same metric objects to be used across different epochs of training or across both training and evaluation.
- You do this with the reset_state() method:

```
def reset_state(self):
    self.mse_sum.assign(0.)
    self.total_samples.assign(0)
```

7.3 Using built-in training and evaluation loops

▣ 7.3.1 Writing your own metrics

- Custom metrics can be used just like built-in ones. Let's test-drive our own metric:

```
model = get_mnist_model()
model.compile(optimizer="rmsprop",
              loss="sparse_categorical_crossentropy",
              metrics=["accuracy", RootMeanSquaredError()])
model.fit(train_images, train_labels,
          epochs=3,
          validation_data=(val_images, val_labels))
test_metrics = model.evaluate(test_images, test_labels)
```



- You can now see the fit() progress bar displaying the RMSE of your model

7.3 Using built-in training and evaluation loops

▣ 7.3.2 Using callbacks

- Running many epochs with model.fit() on a large dataset is a bit like throwing a paper airplane.
- Once launched, you can't control its path or landing point.
- To avoid bad results (and not waste paper airplanes), it's better to use something else instead.
- A drone, for example, senses its environment and adjusts its flight automatically.
- A **Keras callback** turns your model.fit() call from a paper airplane into a smart, self-adjusting drone.
- A callback is an object passed to the model when calling fit() (a class that implements certain methods).
- During training, the model calls the callback at several points.
- **Callbacks can access all information about the model's state and performance, and can handle stopping training, saving the model, loading weights, or reacting to changes.**

7.3 Using built-in training and evaluation loops

▣ 7.3.2 Using callbacks

- Here are some examples of ways you can use callbacks:
  - ➢ Model checkpointing—Saving the current state of the model at different points during training.
  - ➢ Early stopping—Interrupting training when the validation loss is no longer improving (and of course, saving the best model obtained during training).
  - ➢ Dynamically adjusting the value of certain parameters during training—Such as the learning rate of the optimizer.
  - ➢ Logging training and validation metrics during training, or visualizing the representations learned by the model as they're updated—The fit() progress bar that you're familiar with is in fact a callback!

7.3 Using built-in training and evaluation loops

▣ 7.3.2 Using callbacks

- Here are some examples of ways you can use callbacks:
  - ➢ Model checkpointing—Saving the current state of the model at different points during training.
  - ➢ Early stopping—Interrupting training when the validation loss is no longer improving (and of course, saving the best model obtained during training).
  - ➢ Dynamically adjusting the value of certain parameters during training—Such as the learning rate of the optimizer.
  - ➢ Logging training and validation metrics during training, or visualizing the representations learned by the model as they're updated—The fit() progress bar that you're familiar with is in fact a callback!

7.3 Using built-in training and evaluation loops

▣ 7.3.2 Using callbacks

- The keras.callbacks module includes a number of built-in callbacks (this is not an exhaustive list):

```
keras.callbacks.ModelCheckpoint
keras.callbacks.EarlyStopping
keras.callbacks.LearningRateScheduler
keras.callbacks.ReduceLROnPlateau
keras.callbacks.CSVLogger
```

7.3 Using built-in training and evaluation loops

▣ 7.3.2 Using callbacks

- **THE EARLYSTOPPING AND MODELCHECKPOINT CALLBACKS**
  - When training a model, many things are unpredictable.
  - In particular, we don't know how many epochs are needed to get the best validation loss.
  - In past examples, we trained for enough epochs until overfitting started during the first run, then retrained from the beginning using that optimal number of epochs.
  - Obviously, this approach wastes time and resources.
  - A better way is to stop training when the validation loss no longer improves.
  - This can be done using the EarlyStopping callback.

7.3 Using built-in training and evaluation loops

▣ 7.3.2 Using callbacks

- **THE EARLYSTOPPING AND MODELCHECKPOINT CALLBACKS**
  - The EarlyStopping callback stops training when the monitored metric does not improve for a set number of epochs.
  - For example, it can stop training as soon as overfitting starts.
  - This means there's no need to retrain the model with fewer epochs.
  - It is often used together with the ModelCheckpoint callback, which saves the model during training.
  - (Optionally, it can save only the best-performing model so far—the one that achieved the best results at the end of training.)

7.3 Using built-in training and evaluation loops

▣ 7.3.2 Using callbacks

- **THE EARLYSTOPPING AND MODELCHECKPOINT CALLBACKS**

Listing 7.19 Using the `callbacks` argument in the `fit()` method

Callbacks are passed to the model via the callbacks argument in fit(), which takes a list of callbacks. You can pass any number of callbacks.

Interrupts training when improvement stops

Monitors the model's validation accuracy

Interrupts training when accuracy has stopped improving for two epochs

Saves the current weights after every epoch

Path to the destination model file

These two arguments mean you won't overwrite the model file unless val_loss has improved, which allows you to keep the best model seen during training.

You monitor accuracy, so it should be part of the model's metrics.

Note that because the callback will monitor validation loss and validation accuracy, you need to pass validation_data to the call to fit().

```
callbacks_list = [
    keras.callbacks.EarlyStopping(
        monitor="val_accuracy",
        patience=2,
    ),
    keras.callbacks.ModelCheckpoint(
        filepath="checkpoint_path.keras",
        monitor="val_loss",
        save_best_only=True,
    )
]
model = get_mnist_model()
model.compile(optimizer="rmsprop",
              loss="sparse_categorical_crossentropy",
              metrics=["accuracy"])
model.fit(train_images, train_labels,
          epochs=10,
          callbacks=callbacks_list,
          validation_data=(val_images, val_labels))
```

7.3 Using built-in training and evaluation loops

▣ 7.3.2 Using callbacks

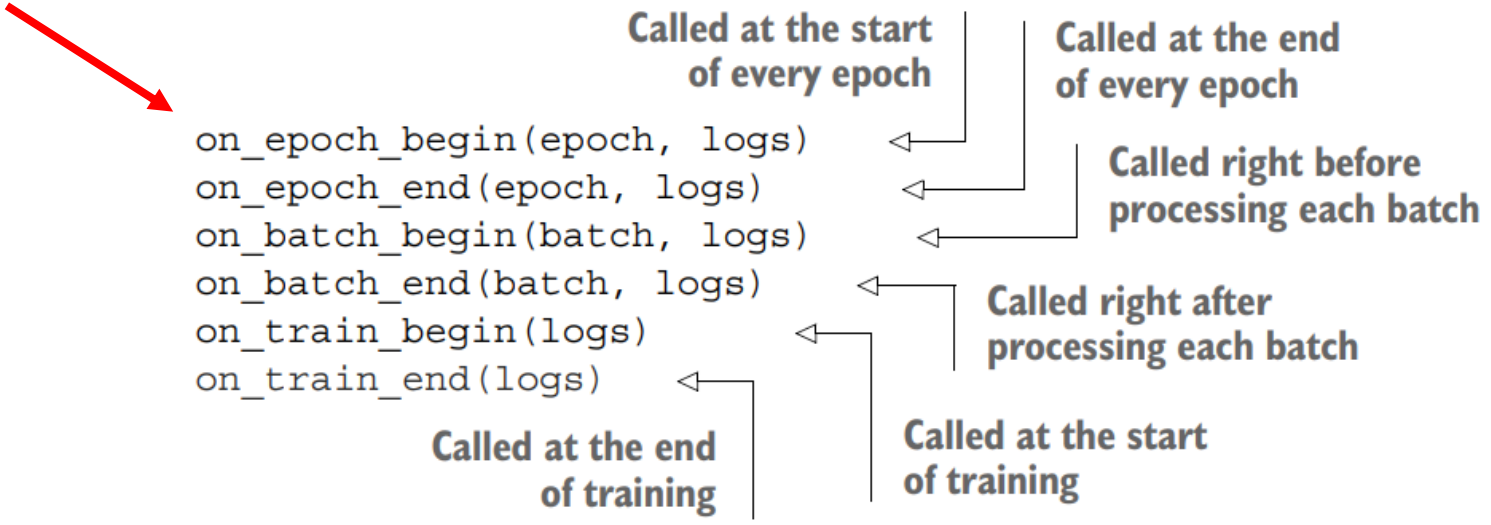
- **THE EARLYSTOPPING AND MODELCHECKPOINT CALLBACKS**
  - ➢ Note that you can always save models manually after training as well—just call model.save('my_checkpoint_path').
  - ➢ To reload the model you've saved, just use

```
model = keras.models.load_model("checkpoint_path.keras")
```

7.3 Using built-in training and evaluation loops

▣ 7.3.3 Writing your own callback

- If you need to take a specific action during training that isn't covered by one of the
- built-in callbacks, you can write your own callback. Callbacks are implemented by subclassing the keras.callbacks.Callback class.
- You can then implement any number of the following transparently named methods, which are called at various points during training:



7.3 Using built-in training and evaluation loops

▣ 7.3.3 Writing your own callback

- All these methods are called with a logs argument.
- This logs argument is a dictionary that contains information about the previous batch, epoch, or training run (such as training and validation metrics).
- Also, the on_epoch_* and on_batch_* methods receive the epoch or batch index as their first argument (an integer).

7.3 Using built-in training and evaluation loops

▣ 7.3.3 Writing your own callback

- Here's a simple example that saves a list of per-batch loss values during training and saves a graph of these values at the end of each epoch.

Listing 7.20 Creating a custom callback by subclassing the `Callback` class

```
from matplotlib import pyplot as plt

class LossHistory(keras.callbacks.Callback):
    def on_train_begin(self, logs):
        self.per_batch_losses = []

    def on_batch_end(self, batch, logs):
        self.per_batch_losses.append(logs.get("loss"))

    def on_epoch_end(self, epoch, logs):
        plt.clf()
        plt.plot(range(len(self.per_batch_losses)), self.per_batch_losses,
                 label="Training loss for each batch")
        plt.xlabel(f"Batch (epoch {epoch})")
        plt.ylabel("Loss")
        plt.legend()
        plt.savefig(f"plot_at_epoch_{epoch}")
        self.per_batch_losses = []
```

7.3 Using built-in training and evaluation loops

7.3.3 Writing your own callback

- Let's test-drive it:

```
model = get_mnist_model()
model.compile(optimizer="rmsprop",
              loss="sparse_categorical_crossentropy",
              metrics=["accuracy"])
model.fit(train_images, train_labels,
          epochs=10,
          callbacks=[LossHistory()],
          validation_data=(val_images, val_labels))
```



7.3 Using built-in training and evaluation loops

▣ 7.3.4 Monitoring and visualization with TensorBoard

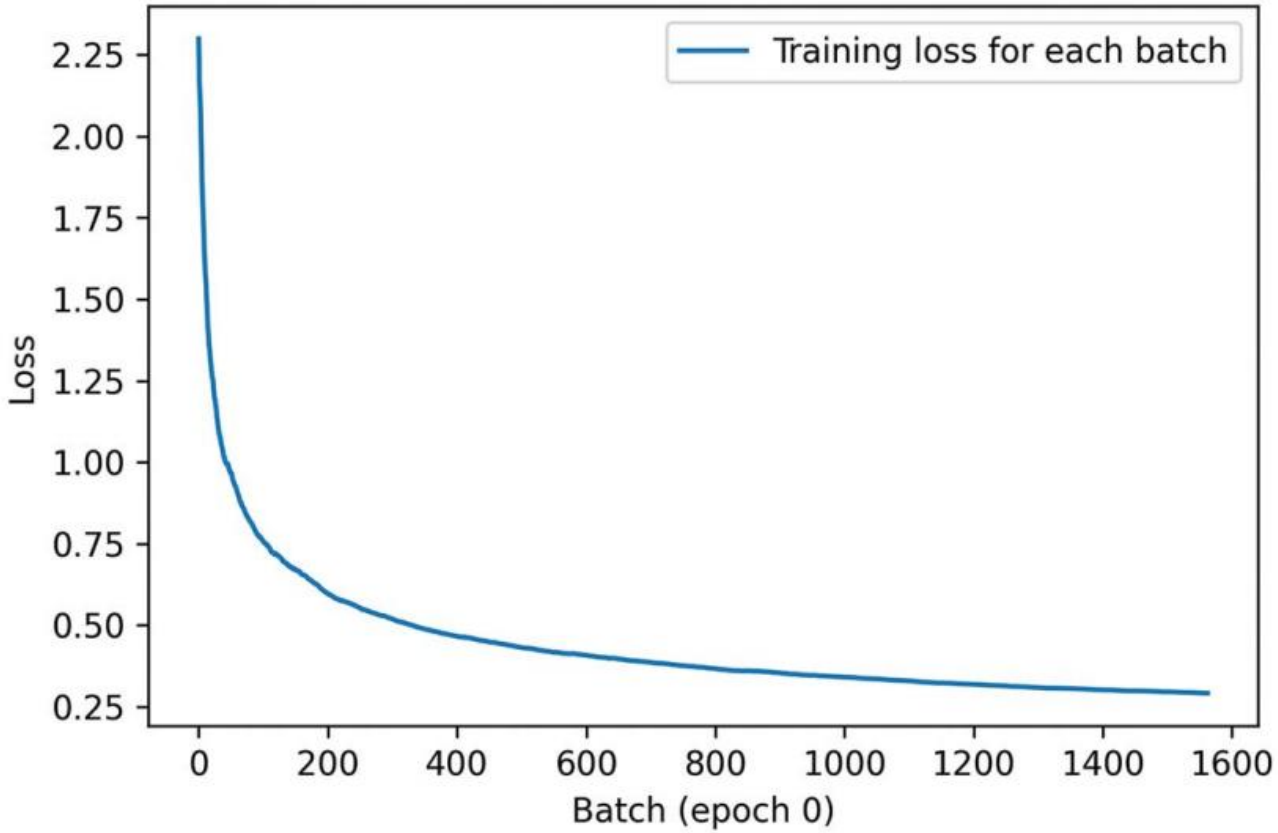


Figure 7.5 The output of our custom history plotting callback

7.3 Using built-in training and evaluation loops

7.3.4 Monitoring and visualization with TensorBoard

- To do good research or build a good model, you must frequently observe and get feedback on what happens inside the model during experiments.
- The purpose of experimentation is to gather as much information as possible about how well the model works.
- Progress happens through repetition, like a loop.
- When you get a new idea, you plan an experiment to test it.
- You run the experiment and process the results.
- These results then inspire the next idea.
- The more you repeat this loop, the more refined and powerful your ideas become.
- Keras helps you turn ideas into experiments quickly.
- With high-speed GPUs, you can get results faster.
- But then, how do we handle these results?
- That's where TensorBoard comes in (see Figure 7-6).

7.3 Using built-in training and evaluation loops

▣ 7.3.4 Monitoring and visualization with TensorBoard

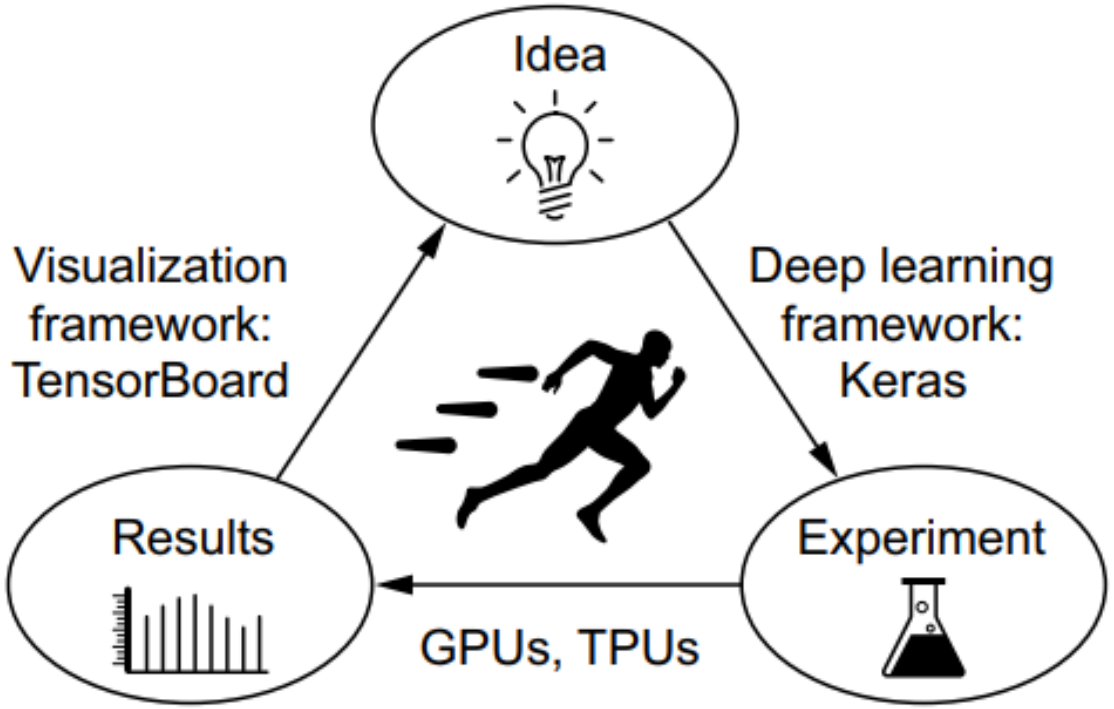


Figure 7.6 The loop of progress

7.3 Using built-in training and evaluation loops

▣ 7.3.4 Monitoring and visualization with TensorBoard

- TensorBoard (https://www.tensorflow.org/tensorboard) is a browser-based application that can run locally.
- It is the best way to monitor everything happening inside a model during training.
- With TensorBoard, you can:
  - ➢ Visually monitor metrics during training
  - ➢ Visualize the model structure
  - ➢ Draw histograms of activations and gradients
  - ➢ Display embeddings in 3D

7.3 Using built-in training and evaluation loops

▣ 7.3.4 Monitoring and visualization with TensorBoard

- By monitoring more than just the final loss, you can get a clearer picture of how the model behaves.
- This helps you improve the model faster.
- The easiest way to use TensorBoard with a Keras model's fit() method is through the keras.callbacks.TensorBoard callback.

7.3 Using built-in training and evaluation loops

▣ 7.3.4 Monitoring and visualization with TensorBoard

- In the simplest case, just specify where you want the callback to write logs, and you're good to go:

```
model = get_mnist_model()
model.compile(optimizer="rmsprop",
              loss="sparse_categorical_crossentropy",
              metrics=["accuracy"])

tensorboard = keras.callbacks.TensorBoard(
    log_dir="/full_path_to_your_log_dir",
)
model.fit(train_images, train_labels,
          epochs=10,
          validation_data=(val_images, val_labels),
          callbacks=[tensorboard])
```

7.3 Using built-in training and evaluation loops

▣ 7.3.4 Monitoring and visualization with TensorBoard

- After the model runs, it will save logs to the specified location.
- If you're running a Python script on your local computer, you can start TensorBoard with the following command:

```
tensorboard --logdir /full_path_to_your_log_dir
```

- After running the command, open the URL shown in the terminal to access the TensorBoard interface.
- If you're using Colab notebooks, you can run a TensorBoard instance directly inside the notebook using a special command.

```
%load_ext tensorboard
%tensorboard --logdir /full_path_to_your_log_dir
```

7.3 Using built-in training and evaluation loops

▣ 7.3.4 Monitoring and visualization with TensorBoard

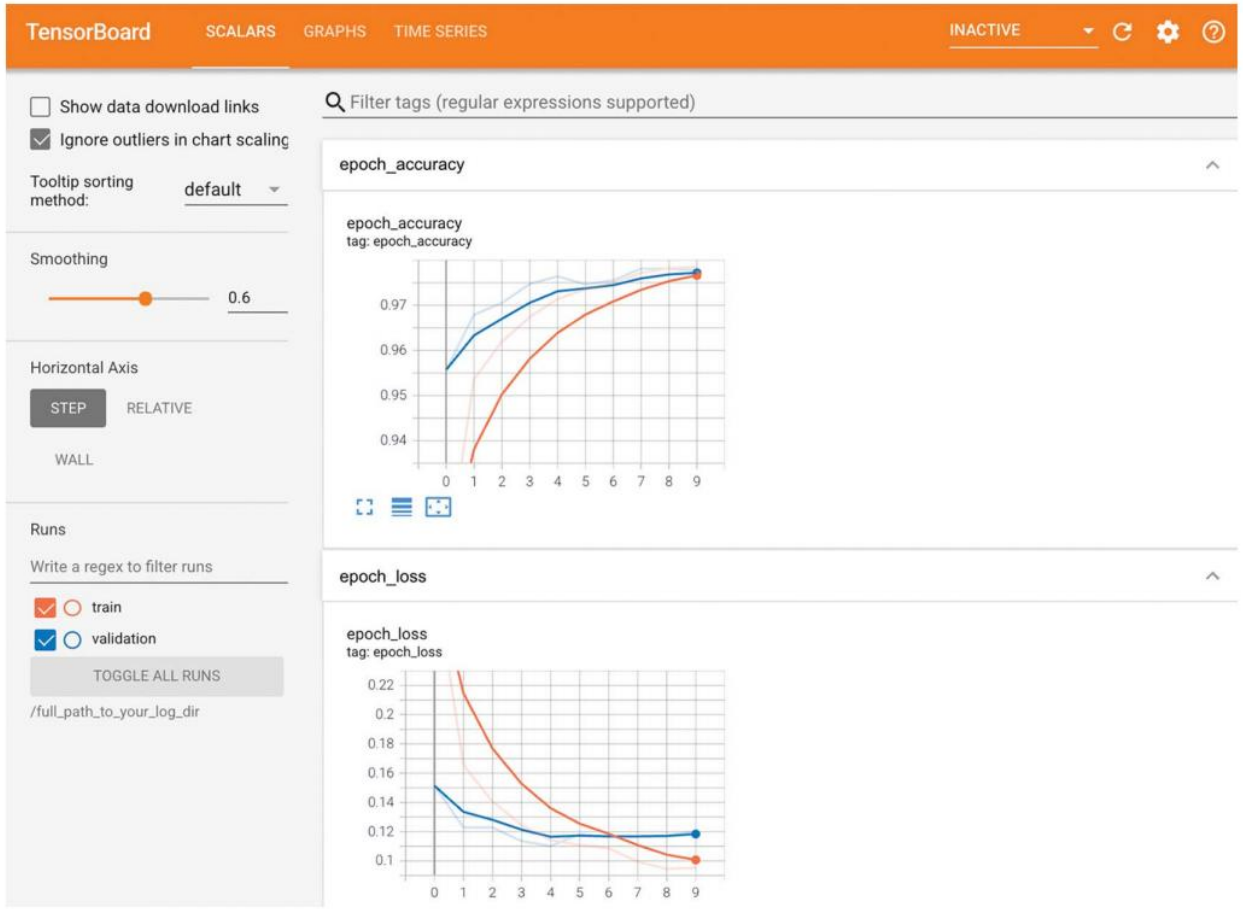


Figure 7.7 TensorBoard can be used for easy monitoring of training and evaluation metrics.

7.4 Writing your own training and evaluation loops

▣ Writing your own training and evaluation loops

- The fit() workflow strikes a good balance between ease of use and flexibility.
- In most cases, you'll use this approach.
- However, even with custom metrics, custom losses, and custom callbacks, it can't cover everything deep learning researchers might want to do.

7.4 Writing your own training and evaluation loops

▣ Writing your own training and evaluation loops

- Above all, the built-in fit() workflow is designed mainly for supervised learning.
- Supervised learning uses input data and related targets (also called labels or annotations) and calculates the loss based on how close the model's predictions are to those targets.
- However, not all forms of machine learning fall into this category.
- For example, there are cases without explicit targets, such as:
  - Generative learning
  - Self-supervised learning (where targets are derived from the input itself)
  - Reinforcement learning (learning through intermittent "rewards," similar to training a dog)
- Even when doing ordinary supervised learning, researchers may want to add new, lower-level flexibility to extend functionality.

7.4 Writing your own training and evaluation loops

▣ Writing your own training and evaluation loops

- When the built-in fit() method isn't enough, you can write your own training logic.
- A typical training loop includes the following steps:
  - ① Run a forward pass inside a gradient tape to compute the model's outputs and the loss for the current batch of data.
  - ② Compute the gradients of the loss with respect to the model's weights.
  - ③ Update the model's weights in the direction that reduces the loss for the current batch.
- These steps are repeated for as many batches as needed.
- This is essentially what the fit() method does internally.

7.4 Writing your own training and evaluation loops

7.4.1 Training versus inference

- In the low-level training loop examples so far, step 1 performs the forward pass, and step 2 extracts the gradients calculated with the gradient tape.
- There are two important details to consider:
  - Some Keras layers, such as **Dropout**, behave differently during **training** and **inference** (when making predictions).
  - Such layers include a **boolean argument training** in their call() method.
- When called with training set to True, Dropout randomly excludes part of the activation outputs.
- When called with training set to False, it does nothing.
- Both Functional and Sequential models also provide a training argument in their call() methods.
- When performing the forward pass, remember to set training to True.

7.4 Writing your own training and evaluation loops

▣ 7.4.1 Training versus inference

- When extracting model weight gradients, you should use model.trainable_weights, not model.weights.
- In fact, layers and models have two types of weights:
  - ➢ Trainable weights: These are updated through backpropagation to minimize the model's loss (e.g., the kernels and biases of a Dense layer).
  - ➢ Non-trainable weights: These are updated only during the forward pass. For example, if you need a custom layer that counts how many batches have been processed, you can store that count in a non-trainable weight and increase it by 1 after each batch.

7.4 Writing your own training and evaluation loops

▣ 7.4.1 Training versus inference

- Among the built-in Keras layers, the Batch Normalization layer (introduced in Chapter 9) is the only one that has non-trainable weights.
- The BatchNormalization layer tracks the mean and standard deviation of the data it processes. It uses these non-trainable weights to approximate feature normalization (a concept introduced in Chapter 6) in real time.

7.4 Writing your own training and evaluation loops

▣ 7.4.1 Training versus inference

- Taking into account these two details, a supervised-learning training step ends up looking like this:

```
def train_step(inputs, targets):
    with tf.GradientTape() as tape:
        predictions = model(inputs, training=True)
        loss = loss fn(targets, predictions)
        gradients = tape.gradients(loss, model.trainable_weights)
        optimizer.apply_gradients(zip(model.trainable_weights, gradients))
```

7.4 Writing your own training and evaluation loops

▣ 7.4.2 Low-level usage of metrics

- In a low-level training loop, you can use Keras metrics—whether built-in or custom.
- You've already learned about the metrics API earlier.
- Simply call update_state(y_true, y_pred) for each batch's targets and predictions.
- Use the result() method to get the current metric value.

```
metric = keras.metrics.SparseCategoricalAccuracy()
targets = [0, 1, 2]
predictions = [[1, 0, 0], [0, 1, 0], [0, 0, 1]]
metric.update_state(targets, predictions)
current_result = metric.result()
print(f"result: {current_result:.2f}")
```

7.4 Writing your own training and evaluation loops

▣ 7.4.2 Low-level usage of metrics

- You may also need to track the average of a scalar value, such as the model's loss.
- You can do this via the keras.metrics.Mean metric:

```
values = [0, 1, 2, 3, 4]
mean_tracker = keras.metrics.Mean()
for value in values:
    mean_tracker.update_state(value)
print(f"Mean of values: {mean_tracker.result():.2f}")
```

- Remember to use metric.reset_state() when you want to reset the current results (at the start of a training epoch or at the start of evaluation).

7.4 Writing your own training and evaluation loops

▣ 7.4.3 A complete training and evaluation loop

- Let's connect the forward pass, backward pass, and metric tracking into a training step function similar to fit().
- This function takes batches of data and targets and returns logs that would normally appear in the fit() progress bar.

Listing 7.21 Writing a step-by-step training loop: the training step function

```
model = get_mnist_model()

loss_fn = keras.losses.SparseCategoricalCrossentropy()
optimizer = keras.optimizers.RMSprop()
metrics = [keras.metrics.SparseCategoricalAccuracy()]
loss_tracking_metric = keras.metrics.Mean()

def train_step(inputs, targets):
    with tf.GradientTape() as tape:
        predictions = model(inputs, training=True)
        loss = loss_fn(targets, predictions)
    gradients = tape.gradient(loss, model.trainable_weights)
    optimizer.apply_gradients(zip(gradients, model.trainable_weights))

    logs = {}
    for metric in metrics:
        metric.update_state(targets, predictions)
        logs[metric.name] = metric.result()

    loss_tracking_metric.update_state(loss)
    logs["loss"] = loss_tracking_metric.result()
    return logs
```

Prepare the loss function.

Prepare the optimizer.

Prepare the list of metrics to monitor.

Prepare a Mean metric tracker to keep track of the loss average.

Run the forward pass. Note that we pass training=True.

Run the backward pass. Note that we use model.trainable_weights.

Keep track of metrics.

Keep track of the loss average.

Return the current values of the metrics and the loss.

7.4 Writing your own training and evaluation loops

7.4.3 A complete training and evaluation loop

- We will need to reset the state of our metrics at the start of each epoch and before running evaluation.
- Here's a utility function to do it.

Listing 7.22 Writing a step-by-step training loop: resetting the metrics

```
def reset_metrics():
    for metric in metrics:
        metric.reset_state()
    loss_tracking_metric.reset_state()
```

7.4 Writing your own training and evaluation loops

7.4.3 A complete training and evaluation loop

- We can now lay out our complete training loop.
- Note that we use a tf.data.Dataset object to turn our NumPy data into an iterator that iterates over the data in batches of size 32.

Listing 7.23 Writing a step-by-step training loop: the loop itself

```
training_dataset = tf.data.Dataset.from_tensor_slices(
    (train_images, train_labels))
training_dataset = training_dataset.batch(32)
epochs = 3
for epoch in range(epochs):
    reset_metrics()
    for inputs_batch, targets_batch in training_dataset:
        logs = train_step(inputs_batch, targets_batch)
    print(f"Results at the end of epoch {epoch}")
    for key, value in logs.items():
        print(f"...{key}: {value:.4f}")
```

7.4 Writing your own training and evaluation loops

7.4.3 A complete training and evaluation loop

- And here's the evaluation loop: a simple for loop that repeatedly calls a test_step() function, which processes a single batch of data. The test_step() function is just a subset of the logic of train_step(). It omits the code that deals with updating the weights of the model—that is to say, everything involving the GradientTape and the optimizer

Listing 7.24 Writing a step-by-step evaluation loop

```
def test_step(inputs, targets):
    predictions = model(inputs, training=False)
    loss = loss_fn(targets, predictions)

    logs = {}
    for metric in metrics:
        metric.update_state(targets, predictions)
        logs["val_" + metric.name] = metric.result()
        loss_tracking_metric.update_state(loss)
        logs["val_loss"] = loss_tracking_metric.result()
        return logs

    val_dataset = tf.data.Dataset.from_tensor_slices((val_images, val_labels))
    val_dataset = val_dataset.batch(32)
    reset_metrics()
    for inputs_batch, targets_batch in val_dataset:
        logs = test_step(inputs_batch, targets_batch)
    print("Evaluation results:")
    for key, value in logs.items():
        print(f"...{key}: {value:.4f}")
```

Note that we pass training=False.

7.4 Writing your own training and evaluation loops

▣ 7.4.3 A complete training and evaluation loop

- We've just implemented the fit() and evaluate() methods!
- They're not exactly the same as the built-in versions.
- The built-in fit() and evaluate() methods provide many additional features, such as large-scale distributed computation.
- Achieving that would require a bit more work.
- They also include several key performance optimizations.

7.4 Writing your own training and evaluation loops

▣ 7.4.4 Make it fast with tf.function

- Although we implemented basically the same logic, you might have noticed that the custom loop runs much slower than Keras's built-in fit() and evaluate() methods.
- This is because TensorFlow code runs eagerly, similar to regular NumPy or Python code—it executes line by line.
- Eager execution makes the code easy to debug, but it is not optimal for performance.

7.4 Writing your own training and evaluation loops

7.4.4 Make it fast with tf.function

- TensorFlow code runs faster when compiled into a computation graph.
- This allows for global optimizations that are not possible in line-by-line interpreted code.
- The syntax to do this is very simple.
- Just add the @tf.function decorator to the function you want to compile before running it.

Listing 7.25 Adding a `@tf.function` decorator to our evaluation-step function

```
@tf.function
def test_step(inputs, targets):
    predictions = model(inputs, training=False)
    loss = loss_fn(targets, predictions)

    logs = {}
    for metric in metrics:
        metric.update_state(targets, predictions)
        logs["val_" + metric.name] = metric.result()

    loss_tracking_metric.update_state(loss)
    logs["val_loss"] = loss_tracking_metric.result()
    return logs

val_dataset = tf.data.Dataset.from_tensor_slices((val_images, val_labels))
val_dataset = val_dataset.batch(32)
reset_metrics()
for inputs_batch, targets_batch in val_dataset:
    logs = test_step(inputs_batch, targets_batch)
print("Evaluation results:")
for key, value in logs.items():
    print(f"...{key}: {value:.4f}")
```

This is the only line that changed.

7.4 Writing your own training and evaluation loops

▣ 7.4.4 Make it fast with tf.function

- On a Colab CPU, running this evaluation loop dropped from 1.8 seconds to 0.8 seconds.
- Much faster!
- When debugging code, it's better not to use the @tf.function decorator and to run in eager execution mode.
- This approach makes tracking bugs easier.
- Once the code works correctly and you want to improve performance, add the @tf.function decorator to the training and evaluation steps.

7.4 Writing your own training and evaluation loops

7.4.5 Leveraging fit() with a custom training loop

- In the previous section, we built a fully custom training loop from scratch.
- This approach provides maximum flexibility, but it requires writing a lot of code and doesn't let you use convenient features like callbacks or distributed training that the fit() method provides.
- What if you need a custom training algorithm, but still want to take advantage of Keras's built-in training features?
- In fact, there's a middle ground between the fit() method and a fully custom training loop.
- You can define your own custom training step function and let the framework handle the rest.
- To do this, override the train_step() method of the Model class.
- This method is called by fit() for each batch of data.
- Then, just call fit() as usual — it will run your custom training algorithm.

7.4 Writing your own training and evaluation loops

▣ 7.4.5 Leveraging fit() with a custom training loop

- Here's a simple example:
  - We create a new class that subclasses keras.Model.
  - We override the method train_step(self, data). Its contents are nearly identical to what we used in the previous section. It returns a dictionary mapping metric names (including the loss) to their current values.
  - We implement a metrics property that tracks the model's Metric instances. This enables the model to automatically call reset_state() on the model's metrics at the start of each epoch and at the start of a call to evaluate(), so you don't have to do it by hand.

7.4 Writing your own training and evaluation loops

▣ 7.4.5 Leveraging fit() with a custom training loop

Listing 7.26 Implementing a custom training step to use with `fit()`

```
loss_fn = keras.losses.SparseCategoricalCrossentropy()
loss_tracker = keras.metrics.Mean(name="loss")

class CustomModel(keras.Model):
    def train_step(self, data):
        inputs, targets = data
        with tf.GradientTape() as tape:
            predictions = self(inputs, training=True)
            loss = loss_fn(targets, predictions)
        gradients = tape.gradient(loss, model.trainable_weights)
        optimizer.apply_gradients(zip(gradients, model.trainable_weights))

        loss_tracker.update_state(loss)
        return {"loss": loss_tracker.result()}

    @property
    def metrics(self):
        return [loss_tracker]
```

This metric object will be used to track the average of per-batch losses during training and evaluation.

We override the train_step method.

We use self(inputs, training=True) instead of model(inputs, training=True), since our model is the class itself.

We update the loss tracker metric that tracks the average of the loss.

We return the average loss so far by querying the loss tracker metric.

Any metric you would like to reset across epochs should be listed here.

7.4 Writing your own training and evaluation loops

▣ 7.4.5 Leveraging fit() with a custom training loop

- We can now instantiate our custom model, compile it (we only pass the optimizer, since the loss is already defined outside of the model), and train it using fit() as usual:

```
inputs = keras.Input(shape=(28 * 28,))
features = layers.Dense(512, activation="relu")(inputs)
features = layers.Dropout(0.5)(features)
outputs = layers.Dense(10, activation="softmax")(features)
model = CustomModel(inputs, outputs)

model.compile(optimizer=keras.optimizers.RMSprop())
model.fit(train_images, train_labels, epochs=3)
```

7.4 Writing your own training and evaluation loops

▣ 7.4.5 Leveraging fit() with a custom training loop

- A Few Important Notes
  - ➢ This pattern works fine with the Functional API, so there's no issue using it to build models that way.
  - ➢ You can use this approach regardless of whether your model is Sequential, Functional, or subclassed.
  - ➢ Since the framework handles it automatically, there's no need to use the @tf.function decorator when overriding the train_step() method.

7.4 Writing your own training and evaluation loops

7.4.5 Leveraging fit() with a custom training loop

- What if we now set the metrics and loss using the compile() method?
- After calling compile(), you can access the following:
  - self.compiled_loss: the loss function passed to compile().
  - self.compiled_metrics: an object containing the list of metrics passed to compile().
  - You can call self.compiled_metrics.update_state() to update all metrics at once.
  - self.metrics: the actual list of metrics passed to compile(). It also includes a metric that tracks loss, similar to what we manually did earlier with loss_tracking_metric.

7.4 Writing your own training and evaluation loops

7.4.5 Leveraging fit() with a custom training loop

- We can thus write

```
class CustomModel(keras.Model):
    def train_step(self, data):
        inputs, targets = data
        with tf.GradientTape() as tape:
            predictions = self(inputs, training=True)
            loss = self.compiled_loss(targets, predictions)
        gradients = tape.gradient(loss, model.trainable_weights)
        optimizer.apply_gradients(zip(gradients, model.trainable_weights))
        self.compiled_metrics.update_state(targets, predictions)
        return {m.name: m.result() for m in self.metrics}
```

Compute the loss via self.compiled_loss.

Update the model's metrics via self.compiled_metrics.

Return a dict mapping metric names to their current value.

7.4 Writing your own training and evaluation loops

7.4.5 Leveraging fit() with a custom training loop

- Let's try it:

```
inputs = keras.Input(shape=(28 * 28,))
features = layers.Dense(512, activation="relu")(inputs)
features = layers.Dropout(0.5)(features)
outputs = layers.Dense(10, activation="softmax")(features)
model = CustomModel(inputs, outputs)

model.compile(optimizer=keras.optimizers.RMSprop(),
              loss=keras.losses.SparseCategoricalCrossentropy(),
              metrics=[keras.metrics.SparseCategoricalAccuracy()])
model.fit(train_images, train_labels, epochs=3)
```

Summary

▣ Summary

- The simplest way to train and evaluate a model is to use the built-in fit() and evaluate() methods.
- Keras callbacks provide an easy way to monitor the model during training and automatically take actions based on its state.
- You can override the train_step() method to fully control how the fit() method behaves.
- You can also write your own training loop from scratch, going beyond fit().
- This is a useful feature for researchers who want to implement completely new training algorithms.



Q & A

Prof. Chang-Hee Han